

UNIDAD 1. DOCKER

1. ¿QUÉ ES DOCKER?

Es una tecnología de virtualización “**ligera**” cuyo elemento básico es la utilización de contenedores en vez de máquinas virtuales y cuyo objetivo es el despliegue de aplicaciones encapsuladas en dichos contenedores.

1.1 La Evolución de Docker

1. **Un único servidor** → Todo se desplegaba en un único servidor
 - **Limitaciones:**
 - Costes elevados
 - Proceso lento
 - Escalado de aplicaciones era costoso y complicado
 - Migración a otro sistema era complicado
 - Se desperdiciaban recursos
 - Había una dependencia del fabricante
2. **Virtualización** → Superaba las limitaciones de un único servidor
 - **Beneficios:**
 - Mejor aprovechamiento de los recursos
 - Procesos de migración y escalado menos complejos
 - Nuevos modelos de negocio en la nube
 - **Inconvenientes:**
 - Cada máquina tiene su propia RAM, almacenamiento y CPU
 - Requiere del arrancado completo del sistema operativo
 - La portabilidad no estaba garantizada al 100%
3. **Contenedores** → Máquinas virtuales ligeras
 - **Características:**
 - Utilizan el mismo Kernel Linux que la máquina física
 - Elimina la sobrecarga del sistema operativo invitado
 - Permiten aislar las distintas aplicaciones
 - Facilitan la distribución de las aplicaciones
 - El Docker Engine apenas añade sobrecarga
 - **Beneficios:**
 - Mayor velocidad de arranque
 - Gran portabilidad
 - Mayor eficiencia
 - **Inconvenientes:**
 - Son más frágiles que las máquinas virtuales

1.2 Docker Sin Sudo

Podemos crear un grupo llamado docker y metemos a los usuarios que queramos que utilicen docker sin necesidades de ser sudo

- **sudo groupadd docker** → Creamos el grupo docker
- **sudo usermod -aG docker \$user** → Añadimos al \$user al grupo docker
- **newgrp docker** → Activamos los cambios tras el reinicio

1.3 Hola Mundo

Esta expresión se utiliza en docker para saber que todo está instalado y funcionando correctamente.

- **docker run hello-world** → Muestra si está todo funcionando

Se ejecutará el contenedor y el mensaje de bienvenida.

1.4 ¿Qué es Docker Hub?

Las imágenes que se descargan en docker, vienen de un registro, un almacén en la nube. Ese almacén se llama **Docker Hub** y cuenta con diferentes beneficios:

- Gran **variedad** de imágenes disponibles
- Permite **crear** y **distribuir** imágenes de manera sencilla
- Permite crear **organizaciones** para equipos y añadir a miembros
- Dispone de una **interfaz web**

2. EJECUTAR Y GESTIONAR CONTENEDORES

- **docker pull nombre_imagen:version** → Descargamos desde el repositorio una imagen con la versión indicada o “latest” si no se indica
 - *docker pull ubuntu:18.04*
- **docker run <opciones> nombre_imagen <comando> <argumentos>** → Arrancamos el contenedor, si no está, descargará la imagen necesaria
 - *docker run -it ubuntu:18.04 /bin/bash*

OPCIONES	DESCRIPCIÓN
-d	Ejecutamos en segundo plano
-e	Establece variables de entorno
-h	Establece el nombre de red para el contenedor
--help	Obtenemos ayuda de las opciones
-i	Mantiene la STDIN abierta del contenedor
--ip	Le damos una IP concreta
--name	Le damos nombre al contenedor
--net	Conectamos el contenedor a una red determinada
-p	Conectamos puertos del contenedor con los del host
--restart	Permite reiniciar un contenedor
--rm	Destruye el contenedor al pararlo
-t	Permite el acceso a un terminal del contenedor
-u	Establece el usuario con el que ejecutamos el contenedor
-v	Monta un bind mount o un volumen en el contenedor
-w	Establece el directorio de trabajo

Ejemplos:

- **docker ps** → Muestra los contenedores en ejecución
- **docker ps -a** → Muestra todos los contenedores creados, estén o no en ejecución
- **docker run -d -p 8888:80 httpd** → Ejecutamos un servidor apache a través del puerto 8888 de mi máquina
- **docker run -d --name servidorDB -p 3306:3306 mysql:8.0.22** → Ejecutamos en segundo plano un contenedor mysql:8.0.22 con el nombre de servidorDB

2.1 Ejecutar Órdenes en Contenedores

Para ejecutar órdenes podemos directamente usar '**run -it /bin/bash**' o podemos utilizar el comando **exec** para ejecutarlo sin necesidad de entrar en el contenedor.

- **docker exec <opciones> nombre_contenedor <orden> <argumentos>**

OPCIONES	DESCRIPCIÓN
-it	Añadimos interactividad con el contenedor
-u	Ejecutamos la orden con el usuario
-w	Ejecutamos la orden desde un directorio concreto

También existe la posibilidad de querer ejecutar un '**cp**' sin necesidad de que entremos en el contenedor

- **docker cp <archivo> <nombre_contenedor:ruta_a_copiar>** → Utilizaremos esto si queremos copiar un archivo local a un contenedor en concreto con una ruta específica
- **docker cp <nombre_contenedor:ruta_a_copiar> <\$directorio/nombre_fichero>** → Usaremos si queremos copiar un fichero del contenedor a una ruta específica de nuestro equipo local

2.2 Obtener información de los Contenedores

- **docker ps** → Nos da la información de los contenedores arrancados
 - **docker ps -a** → Muestra la información de los contenedores creados
 - **docker ps -a -s** → Añade información del tamaño del contenedor
 - **docker ps -l** → Muestra información del último contenedor que se ha creado
 - **docker ps --filter <criterio>** → Filtra los contenedores según el criterio que le digamos
- **docker inspect** → Muestra la información detallada de los contenedores que seleccionemos en formato JSON
 - **docker inspect <nombre_contenedor>** → Inspeccionamos el contenedor por el nombre
 - **docker inspect <ID>** → Inspeccionamos el contenedor por el ID
 - Podemos formatear el texto de salida indicando el criterio que queremos
 - **docker inspect --format 'criterio' <nombre_contenedor>**
- **docker logs** → Nos muestra la información de lo que está ocurriendo en el contenedor
 - **docker logs <nombre_contenedor>** → Muestra los logs del contenedor indicado
 - **docker logs <ID>** → Da la información del contenedor con el ID indicado
 - **-f** → Sigue escuchando la salida que pueden dar los logs
 - **--tail n.º** → Muestras las n.º líneas de los logs

2.3 Gestión de Contenedores

- **docker stop <nombre_contenedor>** → Detiene un contenedor por nombre
- **docker stop <ID>** → Detiene un contenedor por ID
 - **-t n** → Detiene el contenedor en n' segundos
- **docker rm <nombre_contenedor>** → Borra un contenedor por el nombre
- **docker rm <ID>** → Borra un contenedor por el ID
 - **-f** → Fuerza el borrado, ideal para cuando el contenedor esté en ejecución
- **docker start <nombre_contenedor>** → Inicia un contenedor por el nombre
 - **-i** → Inicia el contenedor de modo interactivo
- **docker restart <ID>** → Reinicia un contenedor por el ID
 - **-t n** → Reinicia el contenedor esperando 'n' segundos

3. DESCARGA DE IMÁGENES

Usaremos '**docker pull**' o '**docker run**' para descargar una imagen

- **docker images** → Muestra las imágenes descargadas con la siguiente información
 - *REPOSITORY* → Nombre de la imagen en el repositorio
 - *TAG* → Versión de la imagen
 - *IMAGE ID* → Identificador único de la imagen
 - *CREATED* → Cuando se creo la imagen
 - *SIZE* → Tamaño de la imagen
- **docker pull** → Descargamos la imagen directamente de la nube
 - **-a** → Descargamos todas las versiones de la imagen indicada
 - **-q** → No se muestra la información de las capas al descargarse

3.1 Borrado de Imagen

Conforme avanzamos, vamos acumulando imágenes que no necesitamos en nuestro sistema, para ello podemos utilizar el comando necesario para ir eliminando las imágenes. **Es necesario que el contenedor no esté utilizando la imagen**, si no, deberemos parar el contenedor y luego borrar la imagen.

- **docker rmi <nombre_imagen>** → Borra la imagen según el nombre indicado
- **docker rmi <IMAGE_ID>** → Borra la imagen según el IMAGE ID
- **docker image rm <nombre_imagen>** → Borra la imagen según el nombre indicado
- **docker image rm <IMAGE_ID>** → Borra la imagen según el IMAGE ID
- **docker rmi <nombre1> <IMAGE_ID>** → Borra dos o más imágenes a la vez por el nombre o por el IMAGE ID

OPCIONES	DESCRIPCIÓN
-f	Fuerza el borrado aunque el contenedor esté usando la imagen
prune -a	Borra todas las imágenes que no se esté utilizando
prune --filter criterio	Borra las imágenes que cumplen el criterio del filtro

4. VOLÚMENES DOCKER

Los datos de un contenedor son destruidos si borramos el contenedor, además estos datos no se mueven fácilmente. Para evitar esto, utilizaremos los '**volúmenes**' o los '**bind mount**'

- **Volúmenes** → Conseguimos **persistencia** de los **datos**. Estos datos se almacenan en una parte del sistema de ficheros que es gestionada por **docker**, y sólo docker tendrá acceso.
 - Los datos almacenados son **compartidos** entre los contenedores que utilicen el mismo volumen.
 - **/var/lib/docker/volumes** → Zona reservada en Linux o Mac
 - **/var/snap/docker/common/var-lib-docker/volumes** → Zona reservada en Linux si hacemos un snap
 - **C:\ProgramData/docker/volumes** → Zona reservada en Windows
- **Bind Mounts** → Mapeamos una parte del sistema de ficheros de mi equipo con el sistema de ficheros del contenedor. Con esto logramos que además de docker, otras **aplicaciones** tengan **acceso** a los datos.
 - Se utiliza para la **fase** de **desarrollo** de aplicaciones

4.1 Gestión de Volúmenes

4.1.1 Creación de volúmenes

- **docker volume create <opciones> <nombre_volumen>** → Creamos el volumen

OPCIONES	DESCRIPCIÓN
-d	Especificar el driver elegido
--label	Especificar los metadatos mediante clave-valor
-o	Especificar opciones relativas al driver
--name	Especificamos el nombre para el volumen

4.1.2 Eliminación de volúmenes

- **docker volume rm <nombre_volumen>** → Elimina el volumen según el nombre
- **docker volume rm <ID>** → Elimina el volumen según el ID
- **docker volume rm <nombre1> <nombre2>** → Eliminar dos o varios volúmenes
 - Podemos usar **-f** para **forzar** la eliminación, ya que el contenedor puede estar siendo usado por un contenedor

4.1.3 Obtención de información de volúmenes

- **docker volume ls** → Proporciona una lista de los volúmenes creados y algo de información
- **docker volume inspect <nombre_volumen>** → Da información detallada del volumen según el nombre
- **docker volume inspect <ID>** → Da información detallada del volumen según el ID

4.2 Asociar Almacenamiento a los Contenedores

Podemos utilizar los flags '**-v**' o '**--mount**' en el comando '**docker run**'. Esto **sobreescribirá** la carpeta destino en el sistema de ficheros del contenedor.

- **docker run --name <nombre_contenedor> -v <ruta_local:ruta_contenedor>** → Arrancamos un contenedor dónde la carpeta del contenedor será la carpeta local de nuestro equipo

5. REDES EN DOCKER

- **docker network ls** → Lista las redes existentes
 - Con **-f** → filtramos
- **docker network inspect <nombre_red>** → Lista de manera detallada una red en específico

5.1 Tipos de Redes

- **Bridge** → Driver por defecto. El equipo actúa como puente del contenedor con el exterior
- **Host** → El contenedor usa directamente la red de la máquina
- **Overlay** → Sistema que conecta distintos servicios docker de máquinas diferentes
- **MacVlan** → Asigna una MAC a nuestro contenedor
- **None** → Dejamos al contenedor sin conexión

5.2 Información del Docker Network

- **NETWORK ID** → Identifica la red
- **DRIVER** → Define el tipo de red que se ha utilizado
- **SCOPE** → Indica el ámbito de nuestra red

5.3 Asociar Redes a Contenedores

Una vez hemos creado una red, podemos asociarla con un contenedor usando el comando:

- **docker network connect <nombre_red> <nombre_contenedor>**
- **docker network connect --ip <IP> <nombre_red> <nombre_contenedor>** → Asociamos la red al contenedor con una IP específica

También podemos asociarla en el arranque del contenedor

- **docker run -d --name <nombre_contenedor> --network <nombre_red> <nombre_imagen>**

OPCIONES	DESCRIPCIÓN
--dns	Establece los servidores DNS
--ip6	Establece una dirección IPv6
-h	Establece el nombre del host del contenedor

Para desconectar una red de un contenedor, usaremos:

- **docker network disconnect <nombre_red> <nombre_contenedor>**

6. CONSTRUCCIÓN DE UN CONTENEDOR

Hay 3 formas de construir un contenedor:

1. **Docker commit / docker save / docker load** → Distribución a partir de ficheros
 1. Arrancamos el contenedor desde una imagen base
 2. Realizamos modificaciones en el contenedor
 3. Creamos una nueva imagen usando 'docker commit'
 4. Guardamos en un fichero usando 'docker save'
 5. Distribuimos el fichero
 6. Cargamos el fichero como imagen usando 'docker load'

2. **Docker commit / docker push** → Distribución a través de DockerHub
 1. Arrancamos el contenedor desde una imagen base
 2. Realizamos modificaciones en el contenedor
 3. Creamos una nueva imagen usando 'docker commit'
 4. Nos autentificamos con DockerHub a través de 'docker login'
 5. Distribuimos el fichero subiendo la imagen a DockerHub con 'docker push'
3. **Docker export / docker import** → Distribución a través de ficheros

Comandos

- **docker commit <opciones> <nombre_contenedor> usuarioDockerHub/nombre_imagen**
- **docker save -o <nombre_fichero> usuarioDockerHub/nombre_imagen**
- **docker load -i <nombre_fichero>**
- **docker push <opciones> usuarioDockerHub/<nombre_imagen>**

6.1 Dockerfile

Es un **archivo de texto plano** que contiene una serie de **instrucciones** para crear una imagen de contenedor **Docker**.

- Componentes del **Dockerfile**:
 - **FROM** → Especifica la imagen base
 - **RUN** → Ejecuta comandos para instalar paquete o software dentro de la imagen
 - **COPY / ADD** → Copia o añade archivos desde el sistema local al contenedor
 - **ENV** → Establece variables de entorno para la imagen
 - **CMD / ENTRYPOINT** → Define el comando por defecto que se ejecuta cuando se inicia el contenedor
- **docker build .** → Construimos la imagen estando Dockerfile en el mismo directorio de donde lo ejecutamos
- **docker build -t usuario/nombre_imagen** → Construimos una imagen especificando nombre y estando en el mismo directorio que el Dockerfile

7. DOCKER-COMPOSE.YML

Es un fichero en formato YAML que contiene las instrucciones para crear y configurar los servicios que van a constituir mi aplicación o entorno.

- **Estructura**:
 - **version: 'n'** → Versión del formato
 - **services**: → Lista los contenedores
 - **volumes**: → Volúmenes que usarán los contenedores (opcional)
 - **networks**: → Redes que usarán los contenedores (opcional)

- **Ejemplo → Apache con MariaDB**

version: '3.8'

services:

1. Servicio de Servidor Web (Apache + PHP)

web:

image: php:8.3-apache

container_name: apache-server

Mapea el puerto 80 del contenedor al puerto 8080 en tu máquina

ports:

- "8080:80"

Monta tu código fuente (la carpeta actual .) en el directorio raíz de Apache

volumes:

- ./app:/var/www/html

Asegura que el servicio 'db' esté listo antes de iniciar 'web'

depends_on:

- db

Reinicia si se detiene, a menos que se detenga manualmente

restart: unless-stopped

2. Servicio de Base de Datos (MariaDB)

db:

image: mariadb:10.6

container_name: mariadb-db

Variables de entorno para configurar la base de datos

environment:

CUIDADO: Cambia estas contraseñas en un entorno de producción

MYSQL_ROOT_PASSWORD: root_secret_password

MYSQL_DATABASE: myapp_db

MYSQL_USER: app_user

MYSQL_PASSWORD: user_secret_password

Persistencia de datos: Guarda los archivos de la BD en un volumen

volumes:

- mariadb_data:/var/lib/mysql

restart: unless-stopped

3. Volúmenes nombrados para persistencia de datos

volumes:

mariadb_data:

7.1 Comandos Docker-Compose

COMANDO	DESCRIPCIÓN
docker-compose --version	Versión de docker-compose
docker-compose up	Crea los contenedores del docker-compose
docker-compose up -d	Crea los contenedores en modo detach
docker-compose stop	Detiene los contenedores lanzados
docker-compose run	Inicia los contenedores parados
docker-compose pause	Pause los contenedores
docker-compose unpause	Reanuda los contenedores pausados
docker-compose restart	Reinicia los contenedores
docker-compose down	Para contenedores y los borra
docker-compose down -v	Para contenedores, los borra incluyendo volúmenes